

Deep Learning

Neural networks from the single neuron up: backpropagation, what depth buys you, and the practical knowledge that decides whether a model trains at all.



Utsav Poudel
Founder, Fulcrum

Who made this, and why it is free

Fulcrum is a volunteer-run non-profit. We help people from under-resourced regions learn artificial intelligence through one-on-one mentorship, research and publication supervision, project support, workshops, and a guest speaker series.

The premise is simple. Ability is spread evenly across the world; opportunity is not. What separates a capable student in Pokhara or rural Bihar from one at a well-funded lab is rarely intelligence. It is access to supervision, to reviewers, to equipment, and to the unwritten rules nobody writes down.

This deck is part of how we try to close that. It is free, it will stay free, and you may reuse and adapt it for your own teaching under CC BY 4.0. If it is useful to you, come and find us.

Find us

Website

fulcrumgo.github.io

More material

fulcrumgo.github.io/resources

Discord

discord.gg/gbQCGkupdJ

LinkedIn

linkedin.com/company/gofulcrum

Every deck we publish is free to download at fulcrumgo.github.io/resources

PART ONE

A neuron is a weighted sum and a squashing function

Everything else is arrangement and scale.

From neuron to network

- ▲ One unit computes $z = w \cdot x + b$, then applies a non-linearity $a = \sigma(z)$.
- ▲ Stack units into a **layer**; stack layers into a network. Each layer's output is the next layer's input.
- ▲ Without the non-linearity, stacked linear layers collapse into a single linear layer, depth would buy nothing.
- ▲ The **universal approximation theorem** says one wide hidden layer can approximate any continuous function. It says nothing about whether you can find those weights.

Depth matters in practice because it lets the network reuse intermediate features, so it needs far fewer units than an equivalent shallow model.

Activation functions

- ▲ **Sigmoid** and **tanh** saturate at the extremes, where gradients vanish. This stalled deep networks for years.
- ▲ **ReLU** ($\max(0, x)$) does not saturate for positive input, is cheap, and made deep training practical.
- ▲ **Leaky ReLU** and **GELU** address ReLU's dead-unit problem; GELU is standard in transformers.
- ▲ Output layer is chosen by the task: none for regression, sigmoid for binary, softmax for multi-class.

Backpropagation

Not a learning algorithm, an efficient way to compute gradients.

- ▲ The **forward pass** computes predictions and the loss, caching intermediate values.
- ▲ The **backward pass** applies the chain rule from the loss back to every parameter, reusing those cached values.
- ▲ Cost is roughly the same as the forward pass, which is what makes training large networks feasible at all.
- ▲ Frameworks build a computation graph and do this automatically, but understanding it is what lets you debug a model that will not learn.

Do this once

Implement a two-layer network and its gradients in NumPy, no framework. It takes an afternoon and permanently removes the mystery.

Why deep networks refuse to train

- ▲ **Vanishing gradients**, repeated multiplication by small numbers drives early-layer gradients to zero.
- ▲ **Exploding gradients**, the same mechanism in reverse; fix with gradient clipping.
- ▲ **Poor initialisation**, Xavier and He initialisation keep activation variance stable across layers.
- ▲ **Internal covariate shift**, layer input distributions move as earlier layers update, which normalisation addresses.

Normalisation and residuals

- ▲ **Batch normalisation** standardises activations per mini-batch. Enables higher learning rates, and behaves differently at train and test time, a frequent source of bugs.
- ▲ **Layer normalisation** normalises across features within one example, so it is independent of batch size. Standard in transformers.
- ▲ **Residual connections** ($\text{out} = F(x) + x$) give gradients a direct path backwards and made very deep networks trainable.
- ▲ Together, these three are why 2015-era networks could go deep when 2012-era ones could not.

The main architecture families

- ▲ **MLPs**, fully connected. Fine for tabular data; wasteful on images because they ignore spatial structure.
- ▲ **CNNs**, weight sharing and locality. Efficient wherever position matters and patterns repeat.
- ▲ **RNNs and LSTMs**, process sequences with a carried hidden state. Largely superseded, but still sensible for small-data sequence problems.
- ▲ **Transformers**, attention over a whole sequence at once. Now dominant across text, vision and audio.

Training in practice

- ▲ **Learning rate** is the hyperparameter that matters most. Sweep it over orders of magnitude before tuning anything else.
- ▲ **Schedules**, warmup then cosine decay is a reliable default.
- ▲ **Batch size** trades gradient noise against throughput; larger batches usually need a larger learning rate.
- ▲ **Mixed precision** roughly halves memory and speeds training substantially on modern GPUs.

Debugging order: overfit a single batch. If the model cannot drive loss to near zero on ten examples, you have a bug, not a modelling problem.

Transfer learning

The single most useful technique if you do not have a large dataset or a cluster.

- ▲ Take a model pretrained on a large corpus; its early features are broadly reusable.
- ▲ **Feature extraction**, freeze the backbone, train a new head. Fast, and works with very little data.
- ▲ **Fine-tuning**, unfreeze some or all layers at a low learning rate. Better results, more data needed.
- ▲ **Parameter-efficient methods** (LoRA, adapters) tune a small number of extra weights, making large models adaptable on modest hardware.

Why this matters here

Nearly every Fulcrum mentee is working without institutional compute. Transfer learning and PEFT are what make serious work possible on a free Colab tier.

Reading and reproducing papers

- ▲ Read in this order: abstract, figures, results, then method. Introductions are the least informative part.
- ▲ Ask what the **baseline** is and whether the comparison is fair, same data, same compute, same tuning effort.
- ▲ Check whether code and weights are released. Unreleased results should be treated as provisional.
- ▲ Reproducing a paper is a legitimate contribution, and it teaches more than reading twenty.

ABOUT FULCRUM

Next: Transformers and Attention

The architecture behind most of what you have heard about in the last five years, explained from first principles.

Fulcrum is a volunteer-run non-profit helping people from under-resourced regions learn AI through free mentorship, research supervision, and open material like this deck. Founded 2025 in Kathmandu, Nepal. Nothing we do costs anything.

Website

fulcrumgo.github.io

Every deck, free

fulcrumgo.github.io/resources

Apply for mentorship

discord.gg/gbQCGkupdJ

Partnerships

linkedin.com/company/gofulcrum